



Instituto Tecnológico de San Juan del Río

Ing. Sistemas Computacionales

M A T E R I A:

**TOPICOS DE
CIBERSEGURIDAD**

R002 PARCHADO.EXE

P R E S E N T A:

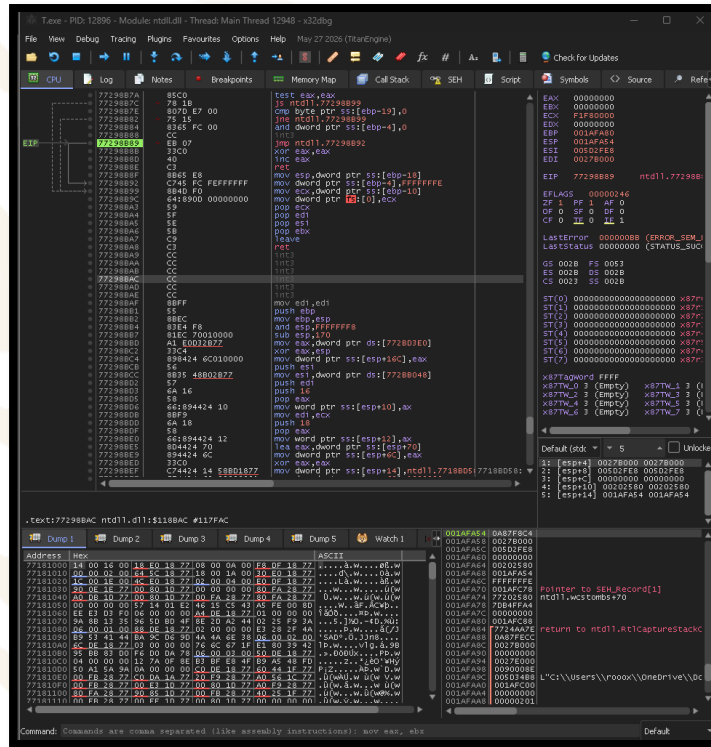
Brian Reynoso Torres. No. 21590295
Gustavo Angel Guerrero Corona No. 21590283
Martín Ortega Centeno No. B20140186

Introducción

En esta práctica trabajamos con un archivo ejecutable que pide un código o contraseña para funcionar. El objetivo principal fue abrir este programa en un depurador para buscar el mensaje de error y encontrar la línea exacta donde se decide si la clave es correcta o no. Al ubicar esa parte, modificamos las instrucciones de ensamblador (cambiando el salto condicional JNE por instrucciones JE o rellenos NOP) para lograr que el programa acepte cualquier clave falsa. Al final, comparamos los archivos para ver qué números cambiaron y programamos un script en Python que hace este mismo cambio de bytes de forma automática.

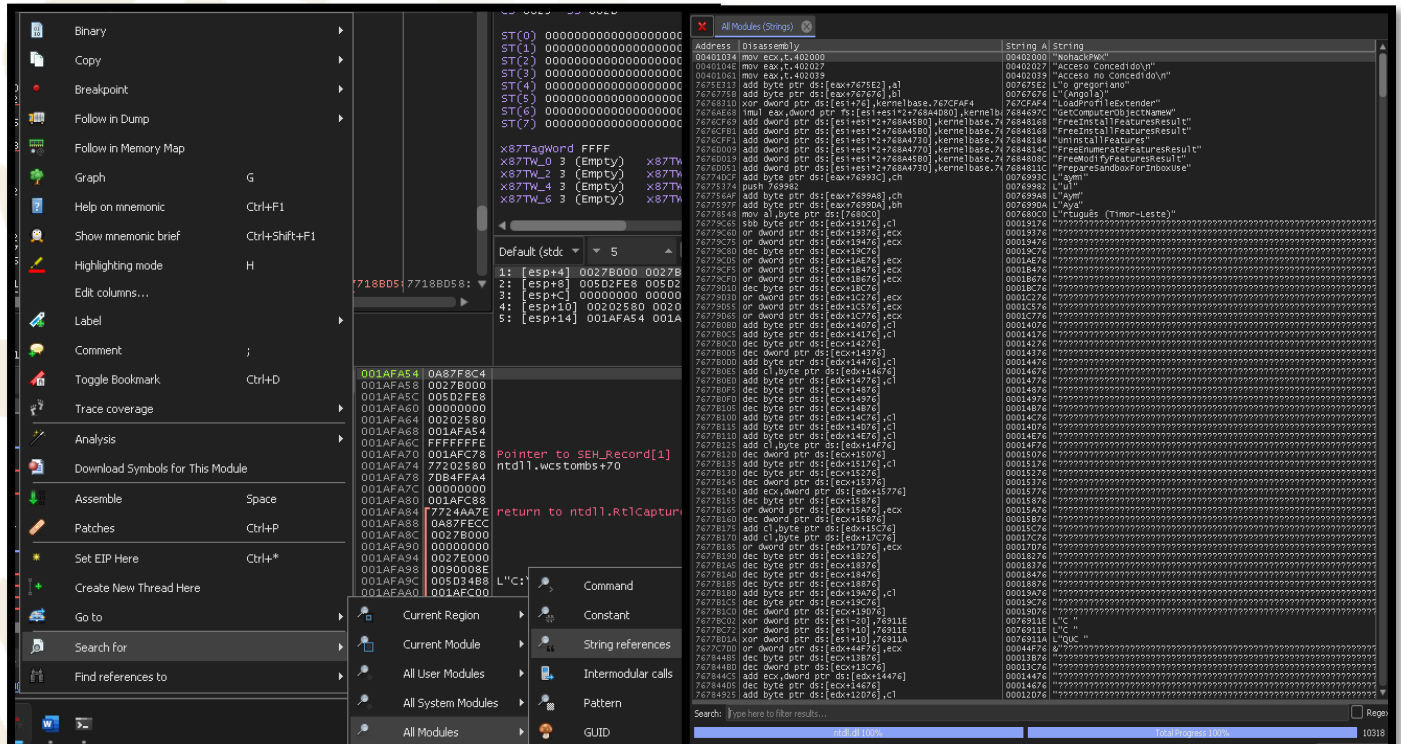
Paso 1

Abrir y cargar archivo binario en el depurador (Debugger).



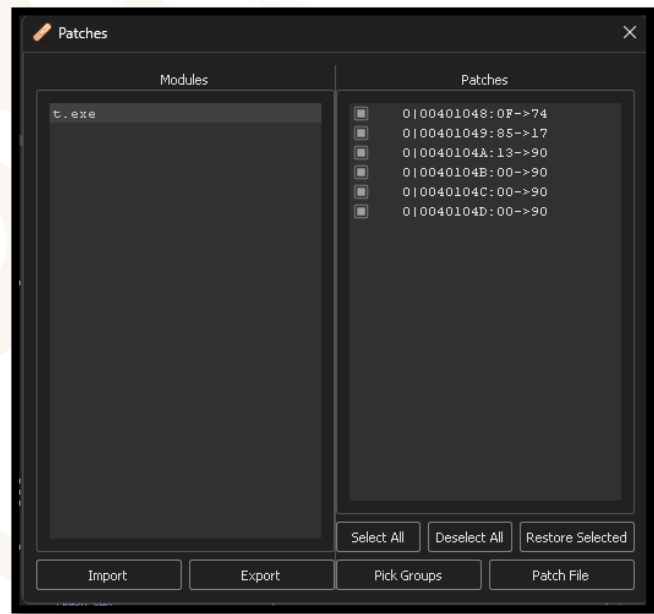
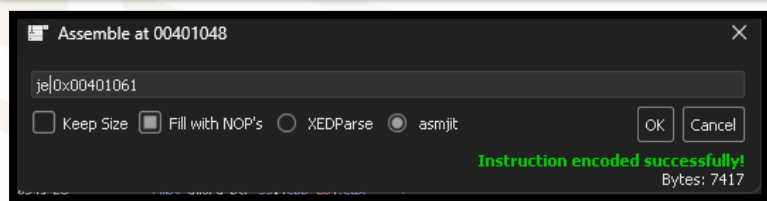
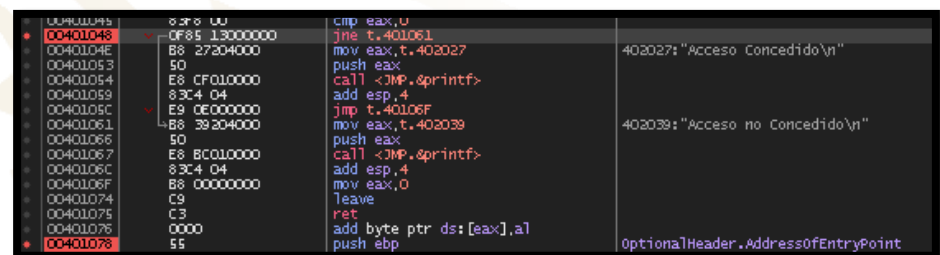
Paso 2

Buscar con Search for las cadenas de texto (Strings) para encontrar el mensaje "Error" o "Acceso".



Paso 3

Modificar el Salto (Parchar en memoria) y guarda el ejecutable modificado.



Paso 4

Comparación hexadecimal y prueba.

```
C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>fc /b "T.exe" "parchado.exe"
Comparando archivos T.exe y PARCHADO.EXE
00000248: 0F 74
00000249: 85 17
0000024A: 13 90
0000024B: 00 90
0000024C: 00 90
0000024D: 00 90

C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>|
```

```
C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>T hola
Acceso no Concedido

C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>parchado hola
Acceso Concedido

C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>|
```

Paso 5

Desarrollar el script de Python para automatizar el parche

```
1  archivo = "T.exe"
2  parche = "prueba_parche.exe"
3
4  posicion = 0x248
5
6  with open(archivo, "rb") as archivo:
7      |  datos = bytearray(archivo.read())
8
9      datos[posicion + 1] = 0x84
10
11  with open(parche, "wb") as archivo:
12      |  archivo.write(datos)
13
14  print("\nParche aplicado correctamente")
15  print(f"\nArchivo creado:", {parche})
16
17
18
```

```
C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>T hola
Acceso no Concedido

C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>prueba_parche hola
Acceso Concedido

C:\Users\rooox\OneDrive\Documentos\TOPICOS DE CIBERSEGURIDAD\Test>
```

Bitácora de Pasos Seguidos (Resumen del Proceso)

- Paso 1: Localización del error. Abrimos el archivo original (T.exe) dentro del depurador x32dbg y rastreamos las cadenas de texto (Strings) en la memoria hasta encontrar el mensaje de bloqueo que decía "Acceso no Concedido".
- Paso 2: Identificación de la llave. Analizamos las líneas de código de ensamblador justo arriba del mensaje de error, localizando la instrucción de comparación (`cmp eax, 0`) y el salto condicional (`jne`) que decide si la clave introducida es correcta o falsa.
- Paso 3: Modificación en memoria. Utilizamos el depurador para alterar ese salto condicional. Al reescribir la instrucción, el depurador modificó de forma automática el bloque reemplazando la instrucción original de 6 bytes por un salto corto optimizado acompañado de bytes de relleno NOP (`90 90 90 90`), logrando así que el programa aceptara cualquier clave falsa.
- Paso 4: Extracción de diferencias binarias. Guardamos el nuevo archivo modificado y abrimos la terminal de Windows (`cmd`). Ejecutamos el comando nativo `fc /b "T.exe" "T_parchado.exe"` para obtener el mapa exacto de los bytes alterados y descubrimos que el cambio iniciaba exactamente en la dirección física (Offset) `0x248`.
- Paso 5: Automatización con Python. Desarrollamos un script de programación muy sencillo en Python que abre el archivo original, lee su contenido de forma binaria, modifica uno por uno los 6 bytes específicos en las posiciones del offset (`0x248` al `0x24D`) y genera automáticamente un nuevo ejecutable completamente parchado.